

THE COMPLETE GUIDE TO PAGINATION

ARCHITECTING CLEAN DATASETS FROM BACKEND TO DB

This document provides a technical blueprint for designing, optimizing, and scaling pagination systems. It explores execution trade-offs across application backends, relational databases, and NoSQL engines.

1. CORE ARCHITECTURAL CONCEPTS

Pagination is the architectural mechanism of breaking a vast dataset into digestible blocks (chunks) to ensure high availability, reduced memory usage, and fast response times. When datasets hit millions of records, querying everything at once can cause significant performance degradation or Out-Of-Memory (OOM) errors.

CORE PERFORMANCE METRICS

Every pagination design must weigh two fundamental operational attributes:

- **Time Complexity:** How does the database search scale as depth increases? (e.g., constant or linear scaling).
- **Data Drift / Drift Consistency:** Handling row mutations (insertions/deletions) gracefully when clients cross data chunks without introducing duplication or skipping records.

2. DEEP-DIVE: OFFSET VS. KEYSET (CURSOR) PAGINATION

There are two primary paradigms used to fetch chunked data across contemporary architectures.

A. OFFSET-BASED PAGINATION

The client requests data by specifying an explicit page number or pointer index (limit and offset). The backend system instructs the database engine to skip the first X records and return the next Y records.

B. KEYSET / CURSOR-BASED PAGINATION

The client requests records sequentially relative to the last evaluation point. The client passes a unique identifying token (a cursor, such as an ID or timestamp) from the previous payload, querying only records that follow that specific cursor.

PROS VS. CONS COMPARISON MATRIX

STRATEGY	PROS 👍	CONS 👎
Offset-Based	• Straightforward implementation. • Supports arbitrary page jumping (e.g., skip directly to Page 45). • Map easily to static UI pagination grids.	• Performance decays linearly at high offsets. • Vulnerable to data drift during high-concurrency writes. • DB must scan and discard all preceding skipped rows.
Keyset (Cursor)	• Highly efficient sequential scanning via indexes. • Immune to data drift duplicate items. • Perfectly matches infinite scrolling and stream feeds.	• Does not support jumping to arbitrary pages out of order. • Requires a unique, deterministic, sequentially ordered column.

3. THE DATABASE LAYER IMPLEMENTATION

A. SQL IMPLEMENTATION (POSTGRESQL / MYSQL)

In relational engines, execution paths differ heavily between scanning and skipping approaches.

⚠️ SQL OFFSET EXAMPLE (SUB-OPTIMAL FOR DEEP DATASETS)

```
-- DB scans and discards 100,000 rows before returning 20 rows
SELECT id, title, created_at
FROM posts
ORDER BY created_at DESC
LIMIT 20 OFFSET 100000;
```

✓ SQL KEYSET / CURSOR EXAMPLE (PERFORMANT INDEX SCAN)

```
-- Utilizing a unique composite index on (created_at, id)
SELECT id, title, created_at
FROM posts
WHERE (created_at, id) < ('2026-06-28 12:00:00', 48291)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

B. NOSQL IMPLEMENTATION (MONGODB)

NoSQL structures scale horizontally; skipping operations can strain distributed shards during coordinator node collation.

✗ MONGODB SKIP APPROACH (HIGH RESOURCE ALLOCATION)

```
db.posts.find().sort({ created_at: -1 }).skip(100000).limit(20);
```

✓ MONGODB RANGE QUERY / CURSOR APPROACH (TARGETED SCAN)

```
db.posts.find({
  created_at: { $lt: ISODate("2026-06-28T12:00:00Z") }
}).sort({ created_at: -1 }).limit(20);
```

4. BACKEND LOGIC: PYTHON IMPLEMENTATION

This implementation details a robust backend abstraction layer for Keyset Pagination using explicit typing and web-safe base64 encoded cursor tokens.

PRODUCTION PYTHON CODE (CURSOR ABSTRACTION)

```
import base64
from typing import List, Dict, Any, Optional

def encode_cursor(timestamp: str, last_id: int) -> str:
    raw_str = f"{timestamp}||{last_id}"
    return base64.b64encode(raw_str.encode()).decode()

def decode_cursor(cursor_str: Optional[str]) -> Optional[Dict[str, Any]]:
    if not cursor_str:
        return None
    decoded = base64.b64decode(cursor_str.encode()).decode()
    ts, record_id = decoded.split("||")
    return {"timestamp": ts, "id": int(record_id)}

def build_paginated_response(items: List[Dict[str, Any]], limit: int) -> Dict[str, Any]:
    has_more = len(items) > limit
    sliced_items = items[:limit]

    next_cursor = None
    if has_more and sliced_items:
        last_item = sliced_items[-1]
        next_cursor = encode_cursor(str(last_item['created_at']), last_item['id'])

    return {
        "data": sliced_items,
        "paging": {
            "has_more": has_more,
            "next_cursor": next_cursor
        }
    }
```


5. ADVANCED ALTERNATIVES & PRODUCTION CONSIDERATIONS



INFRASTRUCTURE ALTERNATIVES

- **Search Engine Delegated Pagination:** Offloading text-filtered queries away from core transactional databases into inverted-index clusters (like Elasticsearch or OpenSearch) using their optimized `search_after` endpoints.
- **Time-Series Bucketing (Redis):** Caching pre-sorted ranges or record identifiers directly inside serialized Redis Sorted Sets (ZSET) keyed against exact epoch Unix timestamps for fast low-latency access layers.



INDUSTRY PITFALLS & GOTCHAS

- **The COUNT(*) Overhead:** Generating total page numbers requires processing high-overhead `SELECT COUNT(*)` full table scans in relational engines. For heavy tables, using internal database metadata statistical estimators prevents disk-read choke points.
- **Opaque Cursors:** Encoding or encrypting cursor arguments protects system internals. Exposing database internal sequential keys or timestamps directly in public URLs creates scraping risks and structural exposure vector liabilities.

```

-- =====
-- OFFSET vs CURSOR (KEYSET) PAGINATION BENCHMARK
-- SQL Server | 500,000 rows | bookings table
-- =====

-- ===== CURSOR (KEYSET) PAGINATION =====

-- Page 1 (no cursor) -> 10ms
SELECT TOP 20 id, flight_no, passenger_name, booking_date, seat, price
FROM bookings ORDER BY id;

-- Page 1,000 (last_id = 19,980) -> 11ms
SELECT TOP 20 id, flight_no, passenger_name, booking_date, seat, price
FROM bookings WHERE id > 19980 ORDER BY id;

-- Page 10,000 (last_id = 199,980) -> 18ms
SELECT TOP 20 id, flight_no, passenger_name, booking_date, seat, price
FROM bookings WHERE id > 199980 ORDER BY id;

-- Page 20,000 (last_id = 399,980) -> 21ms
SELECT TOP 20 id, flight_no, passenger_name, booking_date, seat, price
FROM bookings WHERE id > 399980 ORDER BY id;

-- ===== RESULT =====
-- OFFSET : 9ms -> 35ms -> 57ms -> 138ms    (grows with depth)
-- CURSOR  : 10ms -> 11ms -> 18ms -> 21ms    (stays flat)

```

```
-- =====  
-- OFFSET vs CURSOR (KEYSET) PAGINATION BENCHMARK  
-- SQL Server | 500,000 rows | bookings table  
-- =====  
  
-- ===== OFFSET PAGINATION =====  
  
-- Page 1 (Offset 0) -> 9ms  
SELECT id, flight_no, passenger_name, booking_date, seat, price  
FROM bookings ORDER BY id  
OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY;  
  
-- Page 1,000 (Offset 19,980) -> 35ms  
SELECT id, flight_no, passenger_name, booking_date, seat, price  
FROM bookings ORDER BY id  
OFFSET 19980 ROWS FETCH NEXT 20 ROWS ONLY;  
  
-- Page 10,000 (Offset 199,980) -> 57ms  
SELECT id, flight_no, passenger_name, booking_date, seat, price  
FROM bookings ORDER BY id  
OFFSET 199980 ROWS FETCH NEXT 20 ROWS ONLY;  
  
-- Page 20,000 (Offset 399,980) -> 138ms  
SELECT id, flight_no, passenger_name, booking_date, seat, price  
FROM bookings ORDER BY id  
OFFSET 399980 ROWS FETCH NEXT 20 ROWS ONLY;
```